

MF Analyser / Build Kit

Excel-driven research analytics platform — spec & prompt pack for Claude in VS Code

INTERNAL BUILD DOCUMENT · V1.0

Excel-Driven Research Analytics & Dashboard Module

The master Excel workbook stays the source of analytical truth. This platform ingests it, interprets its logic, executes it, and presents it — so the research methodology can evolve in Excel without rebuilding the application.

Logic-first, presentation-second

React + TypeScript

Python / FastAPI

Workbook-agnostic

01

Overview

What we are building, and the one principle that governs every technical decision.

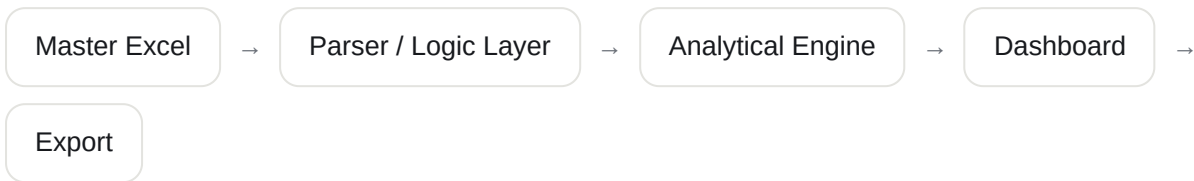
The research team runs its entire analytical process through one master Excel workbook: interconnected sheets holding data inputs, calculations, methodologies, business rules, inter-sheet dependencies, derived metrics, and final research outputs. The workbook works — the problem is operating it, navigating it, and scaling it.

We are **not** rebuilding those calculations in application code. We are building a system in which **Excel defines the analytical logic and the application operationalises and visualises it**. When the team uploads an updated workbook — a new formula, a changed threshold, an extra input column — the platform detects the change, absorbs the new logic, recalculates downstream outputs, and updates the dashboard. No developer in the loop.

THE CRITICAL FRAMING

Do not ask the system to "understand all the Excel logic." Specify an **Excel ingestion + logic interpretation layer** that converts the workbook into a structured representation of *Inputs* → *Formulas* → *Dependencies* →

Transformations → *Business Rules* → *Outputs*. Reading formulas is easy; this structured representation is the actual architectural requirement, and it is what keeps the dashboard from being hard-coded to today's workbook.



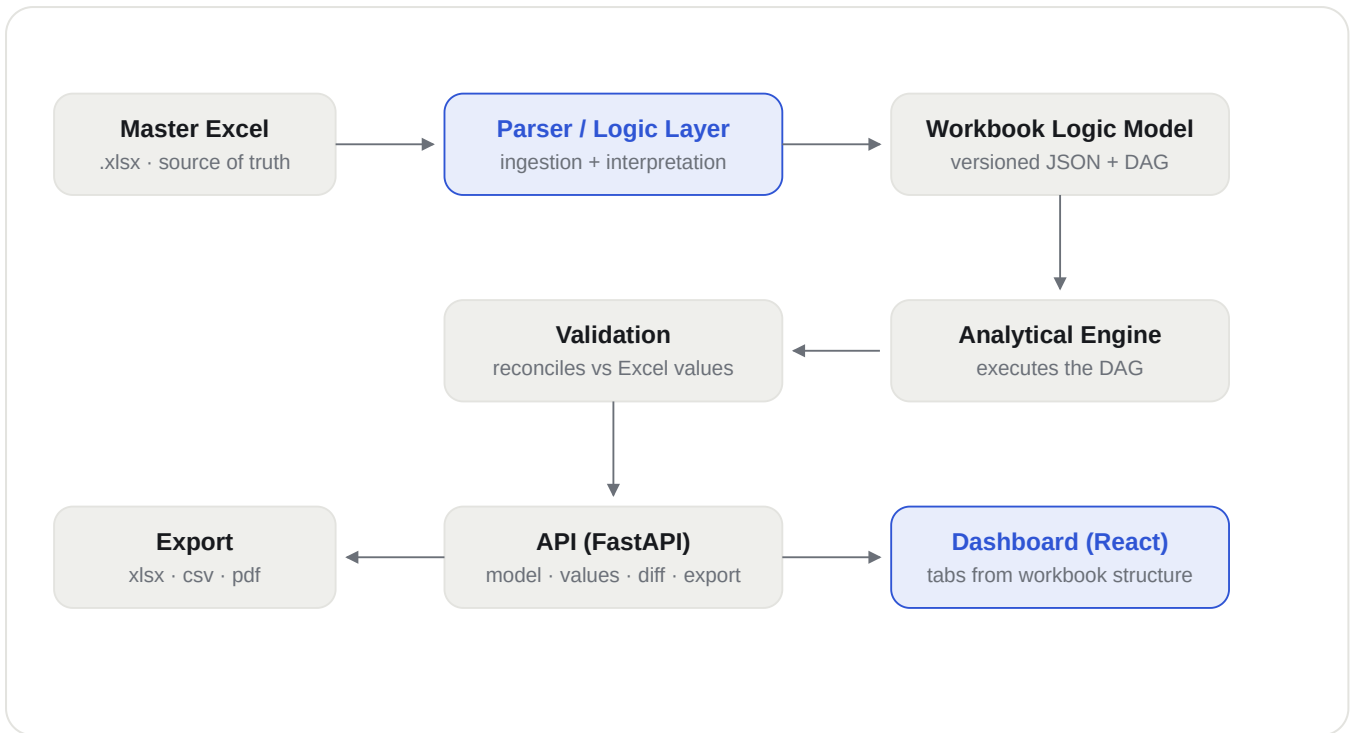
Definition of done

- Upload a workbook → the system builds its logic model, executes it, and renders a tabbed dashboard derived from the workbook's own structure.
- Engine outputs reconcile against the values Excel itself computed, cell by cell, within tolerance — with a visible validation report.
- Upload a *modified* workbook → the system diffs the logic, recalculates affected outputs, and surfaces what changed.
- Users can adjust designated inputs in the dashboard and see recalculated outputs; results export to Excel/CSV/PDF.

02

Architecture

Five layers with strict boundaries. Nothing downstream of the parser may hard-code knowledge of any particular workbook.



Parser layer

Reads the .xlsx with no assumptions about its contents. Extracts sheets, cell values, formulas, named ranges, tables, and formatting hints, and interprets them into the Workbook Logic Model. This is the only layer that ever touches the Excel file format.

Logic model

The structured representation: every cell classified as input, calculation, or output; every formula parsed to an AST; every cross-sheet reference resolved into a dependency graph. Stored versioned, so two workbook uploads can be diffed as *logic*, not as files.

Analytical engine

Executes the logic model — topological order over the dependency DAG, with an Excel-compatible function library. Knows nothing about mutual funds, research methodology, or any specific sheet name.

Validation

Excel caches the last value it computed for every formula cell. After every ingest, the engine's results are reconciled against those cached values cell-by-cell. Mismatches are first-class objects shown in the UI, never silently ignored.

Dashboard

Renders whatever the logic model describes: tabs generated from sheet roles, input panels from input cells, result views from output cells. A schema-driven UI — the workbook shapes it, code doesn't.

THE ONE RULE THAT KEEPS THIS MAINTAINABLE

If any file in the frontend or engine contains a sheet name, a cell address, or a business term from today's workbook, that is a defect. Workbook-specific display

preferences (friendly tab names, chart choices) live in an optional, editable `dashboard.config.json` overlay — never in code.

03

The workbook logic model

The heart of the system: what the parser must produce from any workbook it is given.

Every ingested workbook version is interpreted into one canonical, serialisable model with six parts — matching the required chain **Inputs** → **Formulas** → **Dependencies** → **Transformations** → **Business Rules** → **Outputs**:

COMPONENT	WHAT IT CAPTURES	HOW IT IS DERIVED
Sheet map	Each sheet's role in the analysis: input, staging/transformation, calculation, reference/lookup, output.	Heuristics over cell composition (share of constants vs formulas vs references), plus which sheets feed which — refinable by the user in the UI.
Inputs	Cells and ranges holding manually entered or imported data; their types, units, and valid ranges where inferable.	Constant-valued cells that formulas elsewhere depend on; named ranges; table columns without formulas.
Formulas	Every formula parsed into an AST — function calls, operators, references, literals — not stored as a string to be regex-matched later.	Formula tokenizer over openpyxl-extracted formulas; shared/array formulas expanded per cell.
Dependencies	A directed graph: cell → cells it reads, including cross-sheet edges, named-range edges, and lookup-table edges. Plus the sheet-level rollup of that graph.	Walk each AST's references; resolve names and structured table references; store as a DAG with cycle detection.

Business rules	Conditions, thresholds, classifications, and decision logic: IF/IFS trees, lookup-based classifications, conditional-formatting rules that encode meaning.	Pattern extraction over ASTs (comparison branches, banding via LOOKUP/VLOOKUP/XLOOKUP/INDEX-MATCH), reported in readable form: <i>"Rating = A when score ≥ 0.8"</i> .
Outputs	The cells and ranges that constitute the research result — the terminal nodes users actually consume.	DAG sinks with presentation formatting, cells on output-role sheets, plus user confirmation in the UI.

Two properties are non-negotiable. The model must be **complete enough to execute** — the engine runs from the model alone, never re-reading the file. And it must be **diffable** — comparing two versions yields added/removed/changed inputs, formulas, rules, and outputs, with the changed set traceable to affected downstream outputs through the DAG.

04

Tech stack

Chosen for one reason: Python has the strongest open-source ecosystem for reading, parsing, and executing Excel formula logic.

LAYER	CHOICE	WHY
Backend	Python 3.12 · FastAPI · Pydantic v2	Typed API contracts; async uploads; first-class data tooling.
Excel reading	openpyxl	Reads formulas <i>and</i> cached values (data_only toggle), named ranges, tables, formats — the reconciliation baseline comes free.
Formula engine	formulas (vincilit2000)	Parses formulas to ASTs and builds an executable computation graph — the closest existing thing to our engine. Wrapped behind our own interface, with a custom-function registry as fallback for unsupported functions.

Graph	networkx	Dependency DAG, topological execution order, impact analysis (descendants of a changed node), cycle detection.
Persistence	SQLite → Postgres	Workbook versions, serialised models, run results, validation reports. SQLite is enough to start; the schema stays portable.
Frontend	React 18 · TypeScript · Vite	Schema-driven rendering of tabs and panels from the logic model.
Charts	Recharts	Composable, restrained — fits the minimal visual direction.
Styling	Tailwind CSS + design tokens	Tokens defined once (§06), consumed everywhere.
Export	openpyxl / xlsxwriter · WeasyPrint	Excel and CSV exports of results; print-quality PDF reports.

ENGINE STRATEGY

Phase 2 evaluates the formulas library against the real workbook. If coverage of its function set is high, it becomes the execution core and we keep our own AST layer for interpretation, diffing, and rule extraction. If coverage is poor, the fallback is our AST plus a hand-built evaluator for the function subset the workbook actually uses — the parser reports exactly which functions occur, so the gap is always known and finite.

05

Update lifecycle

The workflow the whole system exists to serve — what happens when a new version of the master workbook arrives.

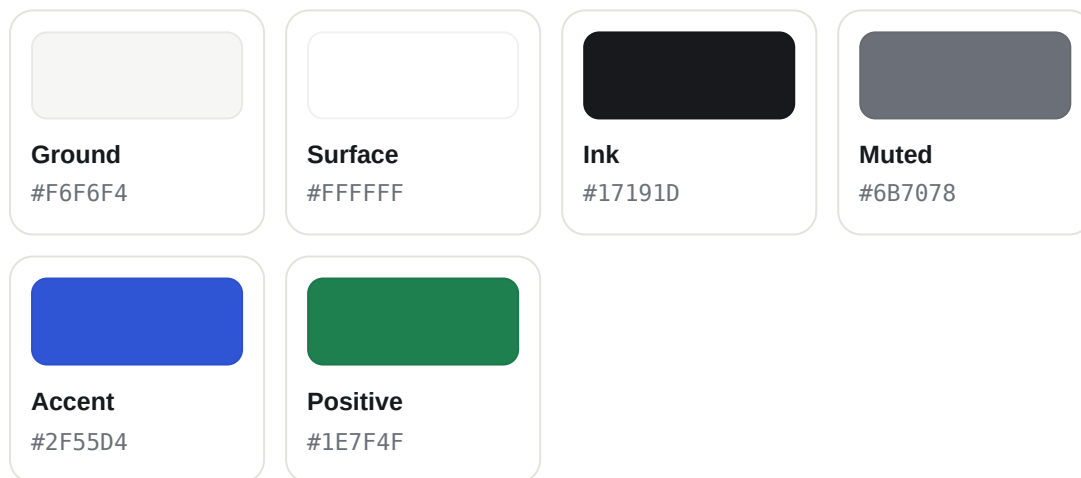


- **Parse & interpret** — the new file becomes a new immutable `WorkbookVersion` with its own logic model. Nothing is overwritten; rollback is switching the active version.
- **Diff** — the change report is expressed in logic terms: *"3 formulas changed on Calculations, 1 new input column on Data, threshold in rating rule moved from 0.75 to 0.80 — affects 14 downstream outputs."*
- **Execute & validate** — full recalculation, then cell-level reconciliation against Excel's cached values. Publication to the dashboard is gated on validation passing (or an explicit, logged override).
- **Publish** — the dashboard re-derives its tabs, inputs, and outputs from the new model. New calculations appear; removed ones disappear; nothing was hand-wired.

06

UI direction

Extremely professional, minimal, quiet. The Apple / CRED register: generous space, precise type, one restrained accent, no noise.



- **Layout** — fixed slim sidebar for module tabs; content on an off-white ground with white cards; 8-pt spacing grid; max content width ~1200px. Density is earned: summary first, detail on demand.
- **Type** — a precise grotesk for headings and UI (Schibsted Grotesk), a clean humanist sans for body (Instrument Sans), tabular figures everywhere numbers align. No more than three sizes per view.
- **Color discipline** — the accent appears only on interactive and selected elements. Semantic green/amber/red is reserved for validation states and metric deltas, never decoration. Everything else is neutral.
- **Surfaces** — hairline borders over shadows; 12–16px radii; one elevation level. Dark mode is a first-class theme with the same tokens, not an inversion.

- **Motion** — 150–200ms ease-out on state changes only. Recalculation gets a quiet progress affordance, not spinners everywhere. Respect prefers-reduced-motion.
- **Numbers** — Indian-format grouping where applicable (12,34,567), explicit units, deltas with direction; a number a researcher will quote is always one click from its lineage (which cells, which formula, which rule).

07

Prompt pack

Run these in order in Claude (VS Code extension), one phase per session. Each ends with acceptance criteria — don't move on until they pass. Phase 0's CLAUDE.md gives every later session its standing context.

PHASE 0 Scaffold & standing context

Set up the monorepo, tooling, and the CLAUDE.md that anchors every later session.

Create a monorepo for an internal "Excel-driven research analytics platform" called mf-analyser.

Structure:

- backend/ – Python 3.12, FastAPI, Pydantic v2, managed with uv (or pip + venv). Packages: app/api, app/parser, app/model, app/engine, app/validation, app/exports, app/storage. Include pytest, ruff, and a pyproject.toml.
- frontend/ – React 18 + TypeScript + Vite + Tailwind CSS + Recharts. Folders: src/api, src/components, src/modules, src/theme, src/lib.
- A root README.md and a Makefile (or justfile) with: dev (runs both servers), test, lint.

Also create a CLAUDE.md at the repo root containing these standing rules for all future sessions:

1. ARCHITECTURE: Master Excel → Parser/Logic Layer → Analytical Engine → Dashboard → Export. The uploaded Excel workbook is the source of analytical truth. The app interprets and executes it; it never re-implements its business logic.
2. The parser converts any workbook into a structured Workbook Logic Model: Inputs → Formulas (as ASTs) → Dependencies (a DAG) → Transformations → Business Rules → Outputs. Everything downstream consumes only this model.
3. HARD RULE: no sheet names, cell addresses, or business-specific terms from any particular workbook may appear in engine or frontend code. Workbook-specific display preferences live only in an editable dashboard.config.json overlay.
4. The engine's results must be reconcilable cell-by-cell against Excel's own cached values; validation is a first-class feature.
5. UI register: minimal, professional, Apple/CRED-like. Off-white ground, white cards, hairline borders, one blue accent (#2F55D4) used only for interactive/selected states, semantic colors only for validation and deltas, Schibsted Grotesk for headings, Instrument Sans for body, tabular figures for numbers, 8pt spacing grid, full dark-mode token set.
6. Write tests alongside every feature; backend features need pytest coverage before they count as done.

Wire up a health-check endpoint and a frontend page that calls it, so `make dev` proves the loop works end to end.

ACCEPT WHEN

Both servers start with one command; frontend shows backend health; lint and an initial test pass; CLAUDE.md is in place.

PHASE 1 Excel ingestion & raw extraction

Read any .xlsx into a faithful raw representation — formulas and cached values both.

Build the ingestion layer in backend/app/parser.

Using openpyxl, load an uploaded .xlsx twice – once with formulas (data_only=False) and once with cached values (data_only=True) – and merge into a RawWorkbook structure (Pydantic models) capturing:

- workbook metadata: sheet names, order, visibility, defined names/named ranges, tables
- per sheet: used range, and for every non-empty cell: address, raw formula (if any), cached value, inferred type (number/text/date/bool/error), number format string, and whether it belongs to a merged range or table
- shared formulas and array formulas expanded so every cell knows its own effective formula

Add:

- POST /api/workbooks – accepts an .xlsx upload, stores the file and its RawWorkbook JSON in SQLite (schema: workbook_versions table with id, filename, uploaded_at, status), returns version id + summary counts (sheets, cells, formula cells, named ranges, distinct Excel functions used, with per-function counts)
- GET /api/workbooks/{id}/raw – returns the RawWorkbook
- The "distinct functions used" report matters: it later tells us exactly which functions the engine must support.

Testing: generate a small fixture workbook in a pytest fixture using openpyxl (inputs sheet, a calc sheet with cross-sheet formulas including IF, SUM, VLOOKUP, a named range) and assert the extraction round-trips correctly. Handle .xlsm gracefully by ingesting cells and flagging "contains macros – macro logic is not interpreted".

ACCEPT WHEN

Fixture tests pass; uploading a real workbook returns accurate counts and a function inventory; cached values are captured for every formula cell.

PHASE 2 Logic interpretation — AST, DAG, classification

Turn the raw extraction into the executable, diffable Workbook Logic Model.

Build the logic interpretation layer in `backend/app/model`, producing a `WorkbookLogicModel` from a `RawWorkbook`.

1. Formula ASTs: parse every formula into an AST (function calls, binary/unary ops, cell refs, range refs, named refs, structured table refs, literals). Evaluate the ``formulas`` library (`vincilit2000/formulas`) first – if its parser is solid, wrap it rather than writing our own tokenizer; either way our AST node types are our own Pydantic models so the rest of the system doesn't depend on the library's internals.
2. Dependency graph: walk each AST's references and build a `networkx DiGraph` of cell-level dependencies, with cross-sheet and named-range edges resolved. Detect cycles (report them; Excel iterative-calc workbooks are flagged unsupported for now). Compute a topological execution order. Also produce the sheet-level rollout graph (which sheets feed which).
3. Cell classification: input (constant with dependents), calculation (has formula), output (formula cell with no dependents, i.e. DAG sink – plus anything on sheets later marked as output sheets), static (constant nothing reads: labels/headers). Associate probable labels with inputs and outputs by scanning adjacent text cells (same row to the left, or column header).
4. Sheet roles: classify each sheet as input / transformation / calculation / reference / output using composition heuristics (share of inputs vs formulas vs external references, in-degree vs out-degree in the sheet graph). Roles are defaults, overridable later via API.
5. Business rules: extract readable rule descriptions from ASTs – IF/IFS condition trees (condition, then, else), threshold comparisons against constants or named cells, and classification bands implied by VLOOKUP/XLOOKUP/INDEX+MATCH against small lookup tables. Store as structured `BusinessRule` objects with source cell references.

Persist the model JSON against the workbook version. Endpoints:

- POST `/api/workbooks/{id}/interpret` → builds and stores the model, returns summary
- GET `/api/workbooks/{id}/model` → full model
- GET `/api/workbooks/{id}/graph?level=sheet|cell` → graph for visualisation
- GET `/api/workbooks/{id}/rules` → extracted business rules

Extend the fixture workbook to cover every classification and rule pattern, and test each.

ACCEPT WHEN

The fixture yields correct classifications, a cycle-free ordered DAG, and readable extracted rules; the real workbook interprets without errors and the sheet graph looks right.

PHASE 3 Analytical engine

Execute the logic model — full runs and what-if recalculation, no Excel required.

Build the analytical engine in `backend/app/engine`.

The engine executes a `WorkbookLogicModel` with no access to the original file:

1. Core: evaluate cells in topological order over the dependency DAG. Preferred path: drive the ``formulas`` library's computation graph, keyed by our model. Fallback path (design the interface so both fit): our own evaluator over our ASTs with an Excel-function registry – implement exactly the functions the Phase 1 inventory reports, and raise `UnsupportedFunctionError` (listing cell + function) rather than silently mis-computing anything.
2. Semantics: match Excel – empty cells coerce to 0 in arithmetic, error values (`#DIV/0!`, `#N/A`, `#VALUE!`) propagate through dependents, text/number coercion follows Excel rules, dates are Excel

serial numbers under the hood.

3. Runs: a Run = model version + input overrides map {cell_or_named_ref: value}. Full run executes everything; incremental run recomputes only descendants of overridden inputs (networkx descendants). Results (all computed cell values + run metadata + timing) persist per run.

4. Endpoints:

- POST /api/workbooks/{id}/runs – body: {overrides?} → run id + results summary
- GET /api/runs/{run_id} – full results
- GET /api/runs/{run_id}/values?sheet=... – computed values for a sheet
- GET /api/workbooks/{id}/lineage/{cellref} – the evaluation tree for one cell: its formula, the values it consumed, and so on recursively (this powers "explain this number" in the UI).

Test against the fixture: full run reproduces every cached value exactly; an override recomputes exactly the affected descendants; error propagation and unsupported-function reporting behave as specified.

ACCEPT WHEN

Fixture runs match Excel's cached values 100%; overrides trigger correct incremental recomputation; lineage endpoint explains any cell.

PHASE 4 Validation & reconciliation

Prove the engine agrees with Excel — the trust layer that gates everything.

Build backend/app/validation.

After interpretation, run the engine with zero overrides and reconcile every formula cell's computed value against the cached value Excel stored in the file:

- numeric: relative tolerance 1e-9 (configurable); text/bool/date: exact; error values must match the same error
- produce a ValidationReport: totals (cells checked, matched, mismatched, skipped-unsupported), per-sheet breakdown, and per-mismatch detail (cell, formula, Excel value, engine value, delta, classification: unsupported function / precision / logic difference)
- status: passed / passed-with-warnings / failed, with thresholds configurable
- a workbook version may only be marked "active" for the dashboard when its latest validation passed, or a user explicitly overrides with a logged reason

Endpoints:

- POST /api/workbooks/{id}/validate → run + report
- GET /api/workbooks/{id}/validation → latest report
- POST /api/workbooks/{id}/activate – enforces the gate, records override reasons

Tests: a fixture where the engine is correct (passes), one with an unsupported function injected (reports skip, not mismatch), and a deliberately perturbed value (fails with correct detail).

ACCEPT WHEN

Reports are accurate on all three fixtures; activation gate enforces validation; the real workbook's report gives a truthful coverage picture.

PHASE 5 Versioning & logic diff

Uploading an evolved workbook becomes a reviewed, explained change — not a mystery.

Build workbook versioning and logic diffing in `backend/app/model/diff.py`.

Diff two `WorkbookLogicModel` versions in logic terms, not file terms:

- sheets added/removed/renamed (renames matched by content-similarity, not just name)
- inputs added/removed; input reclassifications
- formulas added/removed/changed — for changed, an AST-level description ("IF threshold changed from 0.75 to 0.8", "SUM range extended B2:B40 → B2:B60"), falling back to old/new formula text
- business rules added/removed/changed
- named ranges and outputs added/removed/changed
- impact analysis: for every change, the set of downstream outputs affected (DAG descendants), rolled up into a headline: "9 changes affecting 14 of 31 outputs"

Endpoints:

- GET `/api/workbooks/{a}/diff/{b}` → structured `DiffReport`
- POST `/api/workbooks` — extend upload so a new version of the same master automatically parses, interprets, validates, and returns its diff vs the currently active version

Workflow: new versions land as "pending review"; activating one (validation gate from Phase 4) makes it the dashboard's source. Previous versions remain queryable — rollback is re-activating an older version.

Tests: fixture pairs covering each diff category, including a rename plus a threshold change, asserting both the diff detail and the impact set.

ACCEPT WHEN

Diff reports read like a changelog a researcher would trust; impact sets are exact; upload → diff → activate flow works end to end.

PHASE 6 Dashboard shell & design system

The visual foundation — tokens, layout, and navigation derived from the model.

Build the frontend shell and design system (see `CLAUDE.md` rule 5 for the visual register — minimal, Apple/CRED-like, quiet).

1. Theme: define design tokens in `src/theme` (CSS variables consumed by Tailwind): light — ground `#F6F6F4`, surface `#FFFFFF`, ink `#17191D`, muted `#6B7078`, hairline `#E3E3DF`, accent `#2F55D4`, positive `#1E7F4F`, warning `#A16207`, negative `#B4232A`; dark equivalents on `#0E1013` ground with `#16181D` surfaces and a lifted accent `#8CA2EE`. Fonts: Schibsted Grotesk (headings/UI), Instrument Sans (body), tabular-nums wherever digits align. 8pt spacing grid, 12–16px radii, hairline borders instead of shadows, one elevation level. Light/dark follow the system with a manual toggle.
2. Shell: slim fixed left sidebar (workbook name, version badge, module navigation, validation status pill) + main content area, max-width 1200px. Sidebar items and order come from GET `/api/workbooks/{id}/model` sheet roles — nothing hard-coded: Overview, then one entry per module

(grouped sheet roles: Inputs, Analysis, Calculations, Outputs), then Versions, then Validation.

3. Core components (build these first, use them everywhere): Card, StatTile (label, value, delta with direction), DataTable (sticky header, tabular numbers, right-aligned numerics, Indian-format grouping option), Pill (semantic states), EmptyState, SidePanel (for cell lineage later), and a quiet top progress bar for runs in flight.

4. Overview page: workbook name, active version, upload date, counts (sheets, inputs, formulas, outputs), validation status pill, and a sheet-level dependency graph rendered as a clean left-to-right flow (simple SVG layout by topological depth – no heavy graph library).

Populate against the fixture workbook's API responses. No placeholder lorem anywhere; loading, empty, and error states designed, not defaulted.

ACCEPT WHEN

The shell renders entirely from API data; both themes hold up; the sheet-flow diagram reads clearly; components look like one product.

PHASE 7 Module views — inputs, calculations, outputs

The working dashboard: review inputs, run what-ifs, read results, trace any number.

Build the module views, all schema-driven from the logic model.

1. Inputs module: input cells grouped by sheet/section with their inferred labels, current values, and types. Designated inputs are editable (typed controls with validation); edits accumulate into an overrides draft with a clear "n changes · Run analysis / Reset" bar. Running posts overrides to /runs and streams the quiet progress affordance.
2. Calculations module: per calculation sheet, results in DataTables mirroring the sheet's own layout (headers from label detection). Every computed cell opens the lineage SidePanel via /lineage: formula, consumed values, recursively expandable – "explain this number".
3. Outputs module: the research result, presented like a report page, not a grid – output metrics as StatTiles with deltas vs the no-override baseline when overrides are active; ranked/classified lists as DataTables; business rules that produced a classification shown inline in readable form ("Rated A – score $0.84 \geq 0.80$ "). Add charts with Recharts only where the data shape earns them (time series → line, composition → stacked bar); no decorative charts.
4. Versions module: version list with status; the DiffReport rendered as a readable changelog grouped by change type, each entry showing its affected outputs; activate / rollback actions with the validation gate and override-reason dialog.
5. Validation module: the ValidationReport – headline pass/fail, per-sheet coverage, mismatch table with cell, Excel value, engine value, delta.

Keep every screen quiet: summary before detail, accent only on interactive elements, semantic color only for validation and deltas.

ACCEPT WHEN

A researcher can review inputs, run a what-if, see recalculated outputs with deltas, trace any number to its formula, and review a version diff — without touching Excel.

PHASE 8 Exports

Results leave the system as cleanly as they entered.

Build backend/app/exports and the frontend export affordances.

- Excel: export a run's results as .xlsx mirroring the original sheet structure – computed values in place, a cover sheet with run metadata (version, timestamp, overrides applied, validation status). Optionally include original formulas as a second tab-set for auditability.
- CSV: any DataTable view exports what it shows.
- PDF: an "Analysis report" via WeasyPrint – cover, key output metrics, charts, business-rule summary, and the version changelog if overrides or a new version are involved. Typography and palette match the app (print-safe, light theme).
- Endpoints: GET /api/runs/{run_id}/export?format=xlsx|csv|pdf (+ scope params). Frontend: one consistent Export menu on Outputs, Calculations, and Versions views.

Test xlsx round-trip: exported values re-ingest and match the run's stored results.

ACCEPT WHEN

All three formats download correctly; the PDF looks like the product; the xlsx round-trip test passes.

PHASE 9 Hardening & real-workbook QA

Point the finished system at the real master workbook and close every gap it exposes.

QA pass against the real master workbook.

1. Ingest the real master workbook end to end: upload → interpret → validate → activate → dashboard → export. Log every failure or gap as a checklist in QA.md and fix in priority order: (a) validation mismatches, (b) unsupported functions (implement them in the registry), (c) misclassified cells/sheets (improve heuristics, or expose the override UI), (d) label detection misses, (e) UI issues with real data volumes.
2. Performance: interpretation and a full run must feel interactive for a workbook of this size; profile and cache (parsed model reused across runs; incremental runs for overrides). Report timings in QA.md.
3. Robustness: oversized uploads, corrupt files, .xls rejection with a clear message, concurrent runs, empty sheets, merged-cell edge cases.
4. Add an end-to-end test that uploads the fixture, activates it, runs an override, and exports – asserting on the full chain.
5. Update README.md with setup, architecture summary (link CLAUDE.md), and the operational runbook: how the research team uploads a new master version and reviews its diff.

Do not "fix" a validation mismatch by special-casing this workbook's sheet names or cells – improve the general parser/engine, per CLAUDE.md rule 3.

ACCEPT WHEN

The real workbook validates clean (or every residual mismatch is understood and documented); the e2e test passes; a non-developer can follow the runbook.

08

Working in VS Code

How to run this pack with the Claude extension so each phase lands cleanly.

- **Initialise git first.** `git init` before Phase 0, and commit at the end of every phase (phase-1-ingestion, ...). A phase that went sideways is then a cheap revert, not an argument.
- **One phase, one session.** Paste one phase prompt per session/conversation. The CLAUDE.md from Phase 0 is read automatically by the extension, which is how later sessions stay on-architecture without re-pasting context.
- **Hold the acceptance line.** After each phase, ask: *"Run the tests and demonstrate the acceptance criteria for this phase."* If something fails, have it fixed in the same session while the context is warm.
- **Feed it the real workbook early.** From Phase 1 onward, drop the actual master workbook (or a sanitised copy) into a `samples/` folder and add: *"Also run ingestion against samples/master.xlsx and show me the function inventory."* The function inventory from Phase 1 decides how much engine work Phase 3 really needs.
- **Plan mode for the big phases.** For Phases 2, 3, and 7, ask Claude to plan first and review the plan before it writes code — these are the phases with real design decisions in them.
- **When something is off-spec,** quote this document's rule rather than re-explaining: "CLAUDE.md rule 3 — no workbook-specific names in code."

09

Constraints & risks

Known limits of interpreting arbitrary Excel — stated now so they become design decisions, not surprises.

RISK	REALITY	MITIGATION

VBA / macros	Macro logic cannot be interpreted from the file.	Flag on ingest; if the master uses macros for calculation, that logic must move into formulas or be re-specified once, explicitly.
Function coverage	Excel has 500+ functions; no engine supports all.	Phase 1's function inventory bounds the problem to the functions actually used; unsupported ones fail loudly, never silently.
Volatile / external	RAND, NOW, RTD, external links and data connections aren't reproducible.	Detect and report; treat external feeds as declared inputs with refresh, not formulas.
Circular references	Iterative calculation breaks the DAG assumption.	Detect cycles at interpretation; unsupported in v1 with a precise report of the cells involved.
Semantics drift	Excel's coercion and precision quirks are subtle.	This is exactly why cell-level reconciliation (Phase 4) gates activation — correctness is measured, not assumed.
Structural ambiguity	Heuristics will sometimes misread a sheet's role or a label.	Every classification is a default the user can override in the UI; overrides persist across versions.

MF Analyser · internal build document · prepared for the development workflow in Claude / VS Code. The master Excel workbook remains the configurable source of analytical logic.